

JEGYZŐKÖNYV

Webkönyvtárak

Webes adatkezelő környezetek

Órarend Java DOM API-val

Készítette: Kovács Zoárd Gábor

Neptunkód: A6I2XW

Dátum: 2026.05.06.

Sárospatak, 2026

Tartalom

Feladateleírás	3
3. feladat.....	4
3.1 A DOM API működésének bemutatása	4
3.2 XML dokumentum beolvasása	4
3.3 Csomópontok feldolgozása	5
3.4 Konzolos megjelenítés.....	6
3.5 UML diagram készítése	6

Feladateleírás

A gyakorlat célja egy Java nyelven elkészített DOM API alkalmazás fejlesztése volt, amely XML dokumentumok feldolgozására alkalmas. A feladat során a DOM szabvány elemeit kellett alkalmazni egy XML állomány beolvasására, feldolgozására és konzolos megjelenítésére.

A projekt során Eclipse fejlesztői környezetben Java projekt készült, amely a DOM parser segítségével feldolgozza az XML dokumentum teljes szerkezetét. A program feladata a dokumentumban található csomópontok, attribútumok és adatelemek bejárása és blokkos formában történő megjelenítése volt.

A feladat második részében UML osztálydiagram készült a Java alkalmazás és a DOM API kapcsolatainak szemléltetésére. A diagram bemutatja a főbb DOM osztályokat, valamint azok kapcsolatát az elkészített Java programmal.

3. feladat

3.1 A DOM API működésének bemutatása

A DOM (Document Object Model) egy fa-struktúrájú dokumentumkezelési modell, amely lehetővé teszi XML dokumentumok programozott feldolgozását.

A DOM parser működése során:

- az XML dokumentum memóriába töltődik,
- a dokumentum fa-struktúraként reprezentálódik,
- a csomópontok programozottan elérhetők,
- az elemek és attribútumok külön kezelhetők,
- lehetőség van az adatok módosítására és lekérdezésére.

A projekt során az alábbi DOM API osztályok kerültek felhasználásra:

Osztály	Feladat
DocumentBuilderFactory	Parser létrehozása
DocumentBuilder	XML dokumentum beolvasása
Document	XML dokumentum reprezentációja
Element	XML elemek kezelése
Node	Általános csomópont kezelés
NodeList	Gyermekelemek listája
NamedNodeMap	Attribútumok kezelése

3.2 XML dokumentum beolvasása

A program első lépéseként megtörtént az XML dokumentum beolvasása.

A dokumentum feldolgozásához a következő objektumok kerültek létrehozásra:

```
File xmlFile = new File("A6I2XWXML.xml");
```

```
DocumentBuilderFactory factory =
```

```
    DocumentBuilderFactory.newInstance();
```

```
DocumentBuilder builder =  
    factory.newDocumentBuilder();
```

```
Document document = builder.parse(xmlFile);
```

A DocumentBuilderFactory felelős a parser létrehozásáért, míg a DocumentBuilder végzi az XML dokumentum tényleges feldolgozását.

A dokumentum normalizálása:

```
document.getDocumentElement().normalize();
```

A normalizálás biztosítja az XML dokumentum egységes szerkezetét a feldolgozás előtt.

3.3 Csomópontok feldolgozása

A program a DOM fa-struktúrán végighaladva dolgozza fel a dokumentum elemeit.

A gyökérelem gyermekcsomópontjai:

```
NodeList nodeList =  
    document.getDocumentElement().getChildNodes();
```

A program minden csomópont esetén:

- ellenőrzi a csomópont típusát,
- elemként castolja,
- kiolvassa az attribútumokat,
- feldolgozza a gyermekelemeket.

Attribútumok feldolgozása:

```
NamedNodeMap attributes =  
    element.getAttributes();
```

Gyermekelemek feldolgozása:

```
NodeList childNodes =  
    element.getChildNodes();
```

A program kizárólag ELEMENT_NODE típusú elemeket dolgoz fel.

3.4 Konzolos megjelenítés

A feldolgozott XML adatok blokkos formában kerülnek megjelenítésre.

Példa konzolos kimenet:

Csomópont neve: ora

Attribútumok:

id = ora01

tipus = gyakorlat

Adatok:

kurzus: Webkönyvtárak

helyszin: -

oktato: Dr. Patonay Anita

szak: Programtervező informatikus

A blokkos megjelenítés célja az XML dokumentum áttekinthető és könnyen olvasható feldolgozása volt.

Az alkalmazás teljes forráskódja DOM API elemeket használ az XML dokumentum adminisztrálására és feldolgozására.

3.5 UML diagram készítése

A második feladatrész során UML osztálydiagram készült az elkészített Java alkalmazásról.

Az UML diagram bemutatja:

- a saját Java osztályt,
- a DOM API főbb osztályait,
- az objektumkapcsolatokat,
- a használt metódusokat.

UML diagramm:

